Lab 5: Static Code Analysis

1. Issue Table

The table summarizes issues in the original program and the improvements made. Line numbers refer to the original theater_system.py and may differ in the corrected version.

Issue	Type / Tool	Original line(s)	Description	Fix approach
Mutable default argument: system_logs=[]	Code quality — Pylint	9	A mutable default list may be reused across calls, so log entries can persist unexpectedly.	Changed the default to None and initialized a new list inside the function.
Old-style string formatting	Code quality — Pylint	12, 17	Percent-based formatting is less consistent with modern Python conventions.	Replaced percent formatting with f-strings.
Bare except and pass	Error handling — Bandit / Flake8	21	Catching every exception can hide unexpected errors and make failures difficult to diagnose.	Caught the specific ValueError and returned False when a seat could not be removed.
MD5 hash usage	Security — Bandit	25	MD5 is not suitable for security-sensitive hashing and Bandit flags its use.	Replaced MD5 with SHA-256 for the short ticket identifier.
Overly permissive file permissions	Security — Bandit	55	Setting permissions to 0o777 grants read, write, and execute permissions to everyone where applicable.	Removed the os.chmod call instead of granting unnecessarily broad permissions.
File handling without context managers	Code quality / Resource handling	28, 32	Files opened without a with block may not be closed promptly or reliably.	Used with open(..., encoding='utf-8') so files close automatically.
PEP 8 blank-line violations	Style — Flake8	Function boundaries	The original file did not consistently use the required blank lines between top-level functions.	Added the required spacing between functions.
Lines exceeding the recommended length	Style — Flake8	12, 40	Long lines reduced readability and triggered E501.	Split long expressions and print statements across multiple lines.
Non-standard function name	Naming — Pylint / Style	37	printSeatingSummary did not follow conventional snake_case naming.	Renamed it to print_seating_summary and updated the call.
Insufficient seat validation	Program correctness	43–49	The original flow could store out-of-range or non-integer seat values.	Validated integer input and checked that the seat is within the permitted range.

2. Reflection Answers

Q1. What is static code analysis?

Static code analysis examines source code without executing the program. It can identify potential programming errors, security weaknesses, maintainability concerns, and style violations before deployment.

Q2. What are Pylint, Bandit, and Flake8, and how do they differ?

Pylint checks Python code quality, including naming, documentation, possible errors, and maintainability. Bandit scans Python source for common security issues. Flake8 checks style and formatting, including whitespace, blank lines, and line length. Together, they provide complementary checks.

Q3. Why is static code analysis important?

Static analysis helps identify issues early, improve readability and maintainability, and reduce security risks. Fixing problems before deployment can also reduce debugging effort and make code easier to review.

Q4. What issues were found and how were they resolved?

The original program contained a mutable default argument, broad exception handling, MD5 hashing, an overly permissive file-permission change, file-handling concerns, formatting violations, and insufficient seat validation. Fixes included using None for the optional log list, catching a specific exception, switching to SHA-256, removing the chmod call, using context managers, applying PEP 8 formatting, and validating seat inputs.

Q5. What were the final results?

The corrected program achieved a Pylint score of 10/10. The final Bandit scan reported no issues, with zero findings across severity levels. Flake8 produced no output, indicating that it found no style violations in the checked file. The program was also run successfully with valid and invalid seat inputs.

Q6. What did you learn from this experiment?

I learned that code can execute successfully and still contain security, quality, and formatting problems. Pylint, Bandit, and Flake8 detect different categories of issues, so using them together gives a more complete review. I also learned to validate user input, handle exceptions specifically, manage files with context managers, and record changes through Git commits.